Project-Based Assessment Manual

Domain-Specific Generative AI Chatbot Using APIs

VoteEasy: Assistance with voter registration

Slide 1: Title Slide

Project Name: VoteEasy

Tagline: Simplifying Voter Registration in India

Domain: Civic Tech / E-Governance

Technology Stack: Next.js, React, TypeScript, Prisma, SQLite, Groq API

Submitted by: [Aditya Pandey,Sarthak Kumar Sharma,Vivek Chaudhary]

Roll Number: [43,42,44]

Course: INT428 – Generative AI Chatbot Using APIs

Slide 2: Problem Statement

Many citizens, particularly first-time voters, lack proper awareness of voter eligibility criteria, required documents, and the correct procedure to fill Form 6. They often face confusion and errors during registration and find it difficult to track their application status after submission. Existing systems are usually complex, not user-friendly, and provide limited multilingual support. This creates barriers for many users. Therefore, there is a need for a simple, guided, and digital platform integrated with AI assistance to make voter registration more accessible, efficient, and user-friendly.

Slide 3: Objectives

The system aims to provide a comprehensive digital platform for voter registration that simplifies the overall process. It guides users step by step through Form 6, ensuring accurate and error-free submission. The platform enables users to upload required documents, view them in an organized manner, and track their verification status in real time. It also includes a voting-specific AI assistant to answer user queries and provide guidance. Additionally, the system supports multilingual interaction in various Indian languages and offers a personalized dashboard for managing user activities and progress efficiently.

Slide 4: Scope & Limitations

In Scope:

The system includes secure user registration and login functionality, along with a personalized dashboard and profile management features. It provides a guided voter registration process through Form 6 to ensure accuracy and ease of use. Users can store and view document names and track their registration status. The platform also includes an AI-based chatbot limited to voting and

VoteEasy-related queries, along with a basic admin module for reviewing and approving user registrations.

Out of Scope (Current):
The system does not include integration with official Election Commission APIs, cloud-based file storage, real-time SMS or email notifications, or synchronization with official government databases.

Slide 5: Technology Stack

Layer	Technologies Used
Frontend	Next.js 14, React 18, TypeScript, Tailwind CSS
Backend	Next.js API Routes, Prisma ORM
Database	SQLite
Auth & Security	JWT, HTTP-only cookies, bcrypt
AI Integration	Groq API (LLM-based)

Slide 6: System Architecture (High Level)

Frontend Layer: React components + Tailwind UI

Backend Layer: API routes for auth, registration, documents, chatbot, admin

Database Layer: SQLite + Prisma ORM

AI Layer: Groq API with domain-constrained prompts

Data Flow:
User Input → Next.js API → Groq (with prompt + history) → Response → UI

Slide 7: Database Design (3 Core Models)

User Model

id, name, email, password, mobile, language, theme, registrationStatus, registrationData

Chat Model

id, title, createdAt, updatedAt, userId

Message Model

id, role, content, createdAt, chatId

One user → many chats → many messages

Slide 8: Modules Overview (1/2)

User Authentication – Register, login, JWT + HTTP-only cookies

Dashboard – Registration status, document count, AI query count

Voter Registration – Multi-step form (personal, address, docs, review)

Auto Verification – TIMESTAMP-based status change (PENDING → VERIFIED)

Document Management – Shows proof of age, address, photo, verification status

---

Slide 9: Modules Overview (2/2)

AI Assistant – Chat interface, multiple sessions, history, voting-only responses

Multilingual Chatbot – Native script output, user-selectable language

Settings – Profile edit, photo, password change, theme, account delete

Admin Approval – View pending users, approve/reject

Election Deadlines & Eligibility Support – Static reminders + AI guidance

---

Slide 10: AI Assistant – Domain Control (Mandatory Requirement)

Prompt Engineering Strategy:
The system uses carefully designed prompts to control chatbot behavior and ensure domain-specific responses. System instructions strictly limit answers to Indian voting laws, procedures, and VoteEasy platform usage. User registration data is dynamically injected into prompts to provide personalized responses. Additionally, language preferences are included to generate replies in the user's selected Indian language, improving accessibility and user experience.

Model Configuration:

Parameter	Value Used	Justification
Temperature	0.3	Low randomness → factual, deterministic voting answers
Top-p	0.9	Balanced – safe but not overly restrictive
Output Token Limit	500	Enough for helpful replies without verbosity

---

Slide 11: API Integration Evidence

Groq API Integration (Backend):

The backend integrates the Groq API to generate AI-based responses for the chatbot. A structured API call is made with system prompts and conversation history to ensure context-aware replies. The model used is *mixtral-8x7b-32768*, configured for controlled and domain-specific output. Parameters like temperature, top-p, and token limits are carefully set to balance accuracy and response quality.

Response Handling:

Responses are filtered using strict system-level instructions to ensure they remain limited to voting-related and VoteEasy-specific queries. Multilingual output is dynamically enforced based on user preferences, enabling accessible and user-friendly interactions across different Indian languages.

Slide 12: Chatbot Interface Screenshot (Description)

[Actual image would be inserted here]

Shows:

User input in English/Hindi

Chatbot response in selected Indian language (Devanagari or native script)

Dropdown to change language

Session history sidebar

Slide 13: User Workflow (End-to-End)

User signs up / logs in

Dashboard shows registration status

User fills multi-step registration form

Submits → status = PENDING

Auto verification → status = VERIFIED (after configured time)

Documents page displays uploaded proofs

User asks AI assistant voting questions

Admin can approve/reject if needed

Slide 14: Security Measures (Non-Negotiable)

Passwords hashed with bcrypt

JWT stored in HTTP-only cookies

Protected API routes check auth token

Input validation (email, mobile, photo, password)

AI assistant restricted to domain-specific content only

Slide 15: Advantages & Social Impact

Guided, simple voter registration

First-time voter friendly

AI support in multiple Indian languages

Centralized dashboard for all voter tasks

Reduces confusion and errors in Form 6

Civic tech solution with real-world relevance

Slide 16: Future Enhancements

Real file upload (cloud storage)

Election Commission of India API integration

Live deadline updates

OCR-based document verification

SMS / email reminders

Downloadable registration receipt

Full UI multilingual support

Slide 17: Conclusion

VoteEasy successfully demonstrates:

A working full-stack AI chatbot application

Domain-specific constraint (Indian voting)

Use of Generative AI API (Groq)

Model configuration awareness (temperature, top-p)

Document-based Q&A readiness

Multilingual support

Clear problem identification and societal impact

Chatbot-Based Project Evaluation Questionnaire (INT428)

Students Name: Sarthak Kumar Sharma, Aditya Pandey ,Vivek Chaudhary
Roll Number: 42,43,44
Branch & Semester: 324AA(4th)
Project Title: Assistence With voter registration
Guide/Faculty Name: Dr. Jimmy Singla

Section A: Project Overview

Q1. Type of Chatbot Developed

(Select one)

☐ Rule-based

☐ Retrieval-based

☐ Generative (LLM-based)

* Hybrid

Question Note:
This describes the core logic of your chatbot, whether it follows predefined rules, retrieves answers, generates responses using an AI model, or combines multiple approaches.

*The chatbot is hybrid, combining generative AI (LLaMA model) with retrieval-based context for accurate and relevant responses.

Q2. Platform Used for Deployment

(Select all that apply)

* Web Application

☐ Mobile Application

☐ Desktop Application

☐ Messaging Platform (WhatsApp/Telegram/Slack)

☐ Cloud API only (no UI)

Question Note:
Mention where the chatbot is hosted and accessible by users.

Q3. Deployment Link / Access Details

Deployment URL / App Link: https://vote-easy-delta.vercel.app

Question Note:
Provide the live link, GitHub Pages URL, cloud endpoint, or app access details if deployed.

* https://vote-easy-delta.vercel.app

Section B: Model & API Details

Q4. Type of API Used

(Select one)

☐ OpenAI API

☐ Google Gemini API

☐ Azure OpenAI API

* Custom REST API

☐ Local Model API

Question Note:
This refers to the service/API used to communicate with the chatbot model.

*A Custom REST API integrated with Groq is used to communicate with the chatbot model.

Q5. Model Name Used

Model Name: LLaMA 3.3 (llama-3.3-70b-versatile)

Question Note:
Specify the exact model used (e.g., GPT-4, GPT-3.5, LLaMA, Gemini Pro, etc.).

*The model used is LLaMA 3.3 (70B).

Q6. Model Version

Model Version / Release: Model Version / Release: 3.3 (70Bparameters)

Question Note:
Mention the version or release identifier of the model, if available.

*The model used is LLaMA version 3.3 with 70 billion parameters, indicating its specific release and capability level.

Section C: Context & Data Handling

Q7. Contextual Memory Usage

(Select one)

☐ No memory (single-turn chatbot)

☐ Session-based memory

* Long-term memory (database/vector store)

☐ Hybrid memory approach

Question Note:
Contextual memory defines how the chatbot remembers past user interactions.

*Contextual memory allows the chatbot to remember past interactions and use that information to provide more relevant and context-aware responses.

Q8. Flow of Data in the Chatbot

Briefly explain the data flow:

User Input: The user types a message in the React (Next.js) frontend UI.

Backend Processing: The frontend sends the message to the Next.js API route (/api/chat).

Context Retrieval: The backend queries the PostgreSQL database (via Prisma) to retrieve the user's saved profile data, registration status, and the previous chat history for that specific conversation.

Prompt Construction: The backend combines the custom VoteEasy Knowledge Base (Form 6 rules, etc.), the user's specific data, and the chat history into a single structured RAG prompt.

AI Generation: The backend makes a secure REST API call to Groq using the LLaMA model.

Database Save & Response: The AI's response is saved back to the PostgreSQL database, and then sent back to the frontend UI to be displayed to the user.

Question Note:
Describe how user input flows from UI → backend → model → response → user.

*User input is captured in the frontend, sent to the backend via API routes, processed with context data, forwarded to the AI model for response generation, and then returned through the backend to the UI for display.

Section D: Model Configuration & Behavior

Q9. Model Parameters Used

(Fill if applicable)

Parameter	Value	Not Applicable
Temperature	0.5	☐
Top-p / Probability	_____	*
Input Token Limit	_____	*
Output Token Limit	600/15	☐

Question Note:
These parameters control creativity, randomness, and response length of the chatbot, could be discovered from Run settings or Playground settings if using AI studio.

Q10. Thinking Level & Role Assignment

(Select appropriate options)

Thinking Level:

☐ Basic (direct answers)

* Intermediate (context-aware reasoning)

☐ Advanced (multi-step reasoning)

Role Assigned to Model:

☐ Assistant

☐ Tutor

☐ Customer Support Agent

* Domain Expert

Question Note:
Thinking level defines the reasoning depth, while role assignment defines how the chatbot behaves in responses.

*The chatbot uses an intermediate thinking level for context-aware responses and is assigned the role of a domain expert to provide accurate information related to voter registration.

Section E: Technology Stack

Q11. Technology Stack Used

(Mention tools & frameworks)

Frontend: Next.js 14, React 18, TypeScript, Tailwind CSS

Backend: Next.js API Routes, Prisma ORM

Database / Vector Store: PostgreSQL

Cloud / Hosting: Vercel

Question Note:
Specify the complete technical stack used to build and deploy the chatbot.

*The VoteEasy chatbot is built using Next.js, React, and TypeScript for the frontend, Next.js API routes and Prisma for the backend, SQLite/PostgreSQL for database management, Groq API with LLaMA model for AI functionality, JWT and bcrypt for security, and deployed on Vercel.

Section F: Implementation Evidence (Screenshots & Code)

Q12. API Call Screenshot

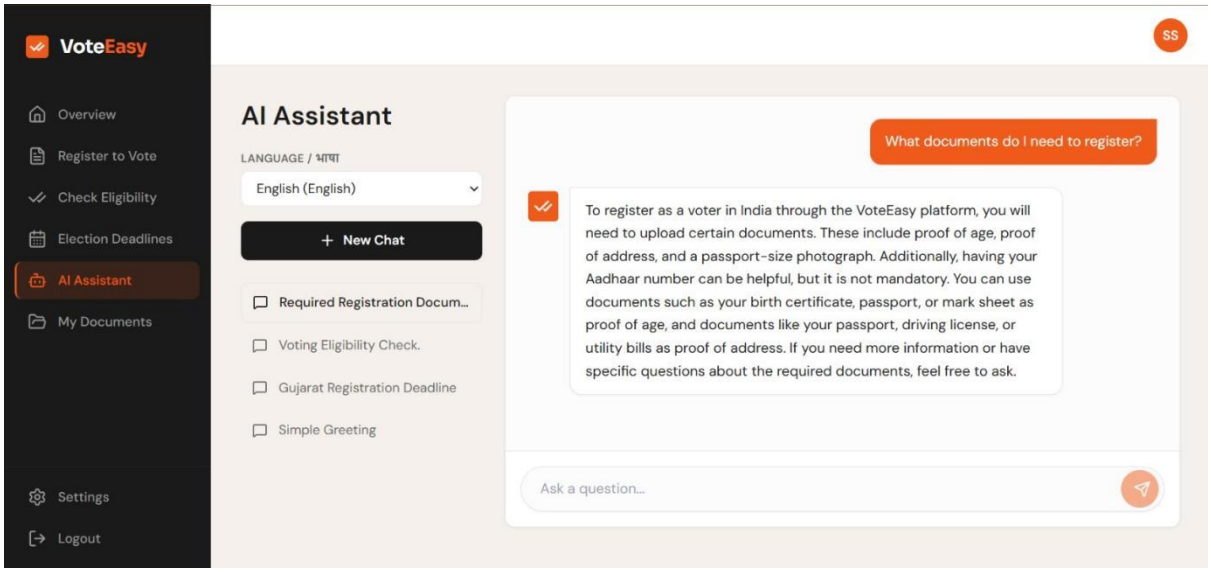
Paste a code snippet showing the chatbot making a successful API request and receiving a response.

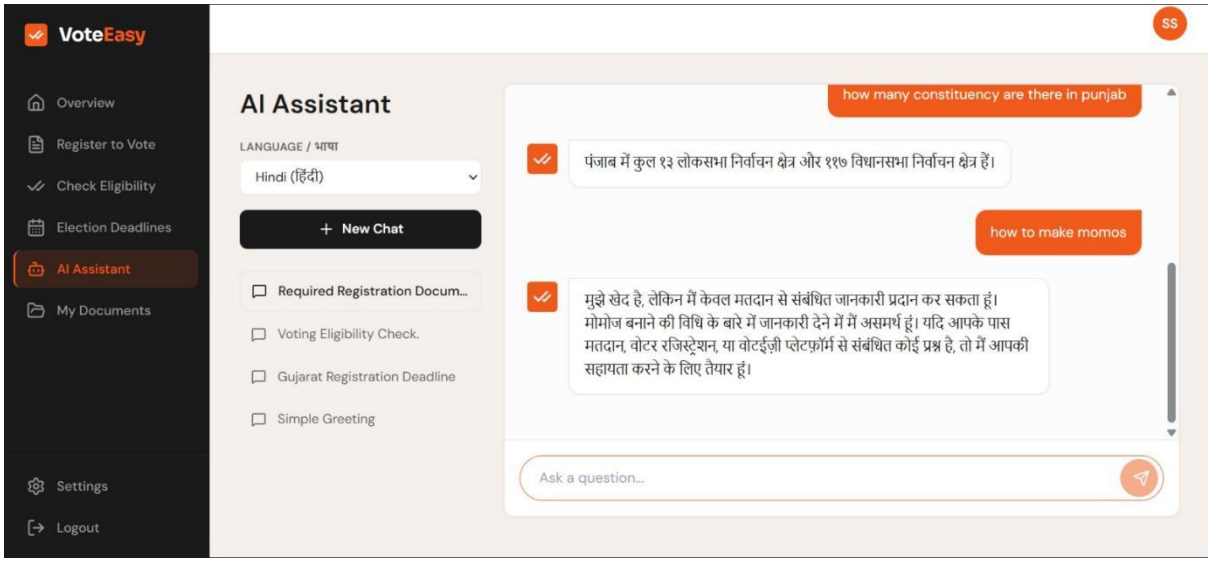
Question Note:
The code snippet should clearly show:

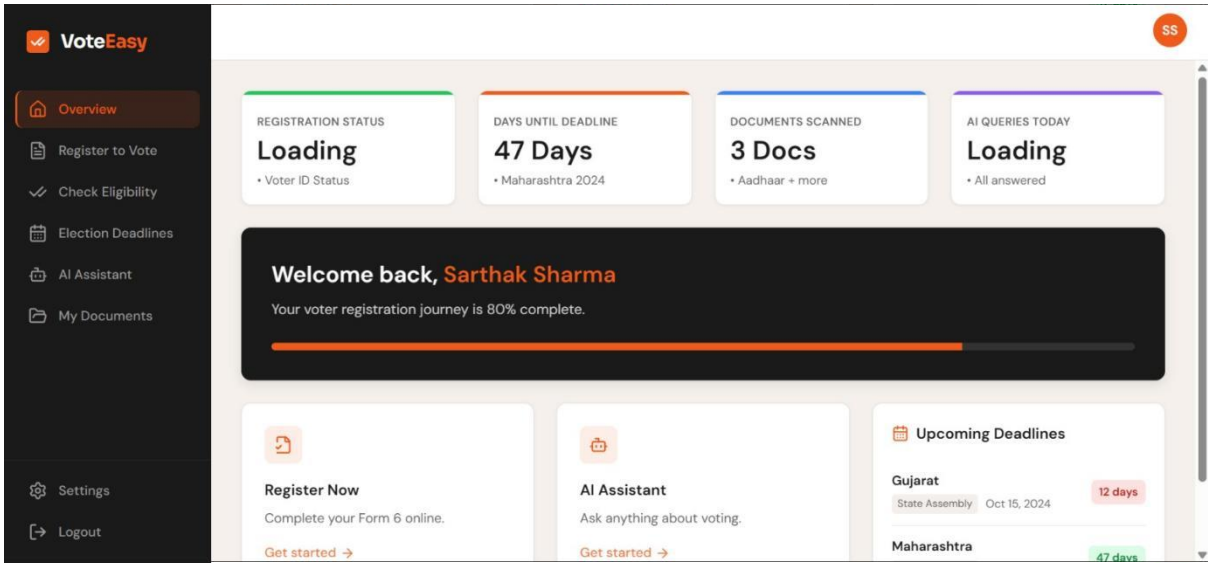
- Model/API response
- User input or prompt
- Successful request execution

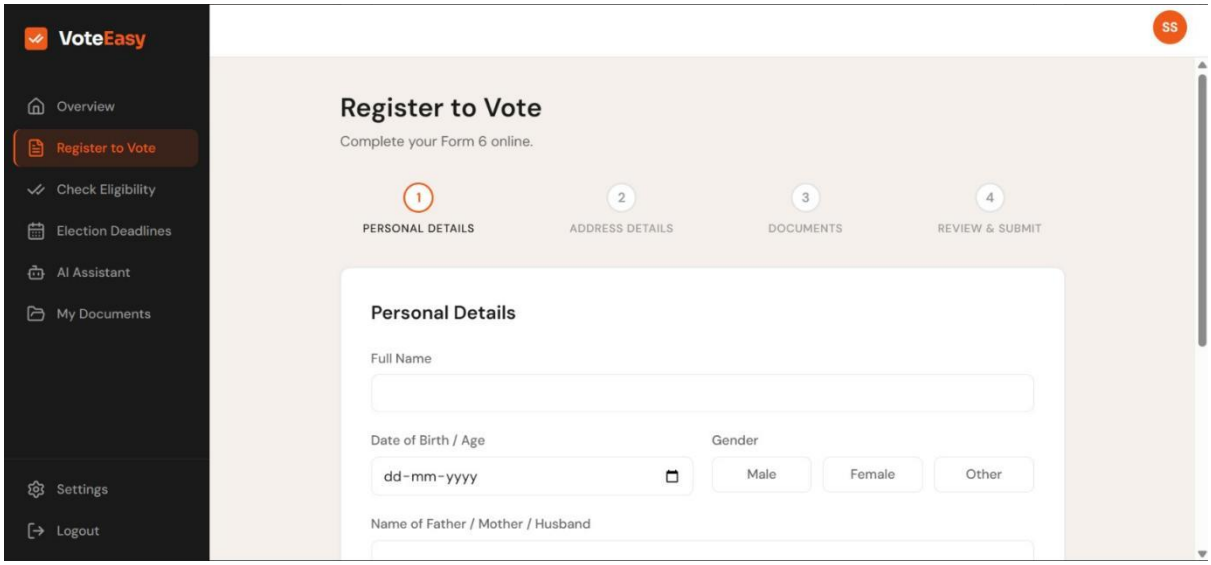
Q13. Chatbot Working Interface Screenshot

Paste a screenshot of the chatbot's user interface while interacting with a user.









Question Note:
The screenshot should show:

- User input
- Chatbot response
- Platform used (web/app/messaging)

Q14. GitHub Repository Link

- Repository URL:
  https://github.com/sarthaksharma05/VoteEasy
- **Question Note:**
  Provide only if the project code is hosted publicly.

Declaration

I confirm that the information provided above is accurate to the best of my knowledge.

Student Signature: Aditya pandey,Sarthak

Kumar Sharma,Vivek Chaudhary
Date: 28/04/2026